

Spark SQL Guide

(sql-programming-guide.html)

JDBC To Other Databases

- Getting Started (sql-getting-started.html)
- Data Source Option (#data-source-option)
- Spark SQL also includes a data source that can read data from other databases using JDBC. This functionality should be preferred over using JdbcRDD (api/scala/org/apache/spark/rdd/JdbcRDD.html). This is because the results are returned as a DataFrame and they can easily be processed in Spark SQL or joined with other data sources. The JDBC data source is also easier to use from Java or Python as it does not require the user to provide a ClassTag. (Note that this is different than the Spark SQL JDBC server, which allows other applications to run queries using Spark SQL).

To get started you will need to include the JDBC driver for your particular database on the spark classpath. For example, to connect to postgres from the Spark Shell you would run the following command:

```
./bin/spark-shell --driver-class-path postgresql-9.4.1207-jars postgresql-9.4.1207.jar
```

Data Source Option

Spark supports the following case-insensitive options for JDBC.

The data source options of JDBC can be set via:

- the `option/.options` methods of `DataSource` or `DataFrameWriter`
- `OPTIONS` clause at `CREATE TABLE USING DATA_SOURCE` (sql-syntax-ddl-create-table-datasource.html)

For connection properties, users can specify the JDBC connection properties in the data source options. user and password are normally provided as connection properties for logging into the data sources.

Property Name	Default	Meaning
url	(none)	The JDBC URL of the form jdbc:protocol:database:URL. e.g., jdbc:postgresql:mydb
table	(none)	The JDBC table that should be used in the query. The table name can be used to specify dbtable and query clause of a SQL query can be used to specify dbtable and query clause.
query	(none)	A query that will be used to read data from the source. Spark will also assign an alias to the query.



(index.html)3.5.1

Spark SQL Guide ([sql-programming-guide.html](#))

- [Getting Started \(sql-getting-started.html\)](#)
- [Data Sources \(sql-data-sources.html\)](#)

- [Generic Load/Save Functions \(sql-data-sources-load-save-functions.html\)](#)
- [Generic File Source Options \(sql-data-sources-generic-options.html\)](#)
- [Parquet Files \(sql-data-sources-parquet.html\)](#)
- [ORC Files \(sql-data-sources-orc.html\)](#)
- [JSON Files \(sql-data-sources-json.html\)](#)
- [CSV Files \(sql-data-sources-csv.html\)](#)
- [Text Files \(sql-data-sources-text.html\)](#)

[Hive Tables \(sql-data-sources-hive-tables.html\)](#)

[JDBC To Other Databases \(sql-data-sources-jdbc.html\)](#)

[Avro Files \(sql-data-sources-avro.html\)](#)

[Protobuf data \(sql-data-sources-protobuf.html\)](#)

[... \(sql-data-sources-...\)](#)

[... \(sql-data-sources-...\)](#)

[... \(sql-data-sources-...\)](#)

[... \(sql-data-sources-...\)](#)

Property Name	Default	Meaning
		Below are a couple of restrictions: 1. It is not allowed to specify 2. It is not allowed to specify required, the subquery can alias provided as part of db Example: <code>spark.read.format("jdbc").option("url", jdbcurl).option("query", "select").load()</code>
	(none)	A prefix that will form the final clause and some databases do queries. As an example, spark w <code><prepareQuery> SELECT <column></code> Below are a couple of examples 1. MSSQL Server does not accept <code>spark.read.format("jdbc").option("url", jdbcurl).option("prepareQuery", "SELECT").load()</code> 2. MSSQL Server does not accept query: <code>spark.read.format("jdbc").option("url", jdbcurl).option("prepareQuery", "SELECT").load()</code>
	(none)	The class name of the JDBC driver
	(none)	These options must all be specified to partition the table when reading from the table in question. Note the number of rows in table. So all rows in the table are read. Example: <code>spark.read.format("jdbc").option("url", jdbcurl).option("dbtable", "(select).option("partitionColumn", "id").option("lowerBound", "1").option("upperBound", "100").option("numPartitions", "10").load()</code>
numPartitions	(none)	The maximum number of partitions to be created. The number of concurrent JDBC connections is limited by <code>coalesce(numPartitions)</code> before the query is executed.
queryTimeout	0	The number of seconds the driver will wait for the query to complete. In the write path, this option limits the timeout of each query instead of the entire write operation.



(index.html)3.5.1

Spark SQL Guide
(sql-programming-
guide.html)

- Getting Started (sql-getting-started.html)
- Data Sources (sql-data-sources.html)
 - Generic Load/Save Functions (sql-data-sources-load-save-functions.html)
 - Generic File Source Options (sql-data-sources-generic-options.html)
 - Parquet Files (sql-data-sources-parquet.html)
 - ORC Files (sql-data-sources-orc.html)
 - JSON Files (sql-data-sources-json.html)
 - CSV Files (sql-data-sources-csv.html)
 - Text Files (sql-data-sources-text.html)
 - Hive Tables (sql-data-sources-hive-tables.html)
 - JDBC To Other Databases (sql-data-sources-jdbc.html)
 - Avro Files (sql-data-sources-avro.html)
 - Protobuf data (sql-data-sources-protobuf.html)

Property Name	Default	Meaning
fetchsize	0	The JDBC fetch size, which defaults to low fetch size (e.g. Or
batchSize	1000	The JDBC batch size, which defaults to 1000. This option applies only to writing.
isolationLevel	READ_UNCOMMITTED	The transaction isolation level, which can be READ_UNCOMMITTED, REPEATABLE_READ, or SERIALIZABLE. The default is READ_UNCOMMITTED. P
sessionInitStatement	(none)	After each database session is created, the sessionInitStatement option (or a PL/SQL block). The default is none.
truncate	false	This is a JDBC writer related option instead of dropping and recreating tables. However, it will not work in some databases. To use the truncate option to use DROP TABLE, use this. MySQLDialect, DB2Dialect, and JDBCDirect doesn't. For
cascadeTruncate	the default cascading truncate behaviour of the JDBC database in question, specified in the JDBC dialect	This is a JDBC writer related option. It allows execution of a TRUNCATE TABLE to prevent inadvertently truncating data.
createTableOptions		This is a JDBC writer related option for creating a table (e.g., CREATE TABLE).
createTableColumnTypes	(none)	The database column data types. The default is none. The same format as CREATE TABLE is used. Valid spark sql data types.
customSchema	(none)	The custom schema to use for specifying partial fields, and the default is none. It is identical to the corresponding schema using the defaults.
pushDownFilter	true	The option to enable or disable pushing down filters to the JDBC data source and thus all filters are performed faster by Spark than
pushDownAggregate	true	The option to enable or disable pushing down aggregates to the JDBC data source. Aggregate push-down is usually enabled. Note that aggregates can be pushed down only if numPartitions equals to 1 or the aggregate is completely and not apply a fine-grained source output.
pushDownLimit	true	The option to enable or disable pushing down the Top N operator. The default is true. Otherwise, if sets to false, LIMIT is not pushed down. Spark still applies LIMIT or LIMIT



(index.html)3.5.1

Spark SQL Guide (sql-programming-guide.html)

- Getting Started (sql-getting-started.html)
- Data Sources (sql-data-sources.html)

keytab

Load/Save

Functions (sql-

data-sources-

principal

load-save-

functions.html)

◦ Generic File

Source Options

(sql-data-

sources-

generic-

options.html)

Per-Krb5Config

(sql-data-

sources-

parquet.html)

◦ ORC Files (sql-

data-sources-

orc.html)

◦ JSON Files (sql-

data-sources-

json.html)

◦ CSV Files (sql-

data-sources-

csv.html)

◦ Text Files (sql-

data-sources-

text.html)

◦ Hive Tables

(sql-data-

sources-hive-

tables.html)

◦ JDBC To Other

Databases (sql-

data-sources-

jdbc.html)

◦ Use of Data Lake

(sql-data-

sources-

avro.html)

◦ Protobuf data

(sql-data-

sources-

protobuf.html)

◦ MS SQL

◦ Oracle

◦ PostgreSQL

Property Name	Default	Meaning
		if LIMIT or LIMIT with SORT is pushed down to the data source.
pushDownOffset	true	The option to enable or disable push down OFFSET to the JDBC data source. If pushDownOffset is true and numLimit is not set, the query will not be pushed down and Spark will execute the query on the driver.
pushDownTableSample	true	The option to enable or disable push down TABLESAMPLE to the data source.
keytab	(none)	Location of the kerberos keytab for the JDBC client. When path is set, both keytab and principal are required.
principal	(none)	Specifies kerberos principal name for authentication.
refreshKrb5Config	false	This option controls whether the JDBC driver refreshes the krb5.conf file on the driver side. Set to true if you want to refresh the krb5.conf file on the driver side. This option is only effective when the keytab and principal are set. 1. refreshKrb5Config flag is set to true 2. A JDBC connection is provided 3. The krb5.conf is modified by the JDBC driver 4. Spark authenticates successfully 5. The JVM loads security configuration 6. Spark restores the previous configuration 7. The modified krb5.conf configuration is used for the next connection
connectionProvider	(none)	The name of the JDBC connection provider. Used to specify the connection provider. The provider must not be disabled in the JDBC driver.
preferFilesWithNTZ	false	When the option is set to true, the JDBC driver will prefer files with NTZ (Not a Time Zone) over files with TZ (Time Zone) for TIMESTAMP data.

Note that Kerberos authentication with keytab is not always supported by the JDBC driver.

Before using keytab and principal configuration options, please make sure the following requirements are met:

- The included JDBC driver version supports kerberos authentication with keytab.
- There is a built-in connection provider which supports the JDBC driver.
- Use of Data Lake.

There is a built-in connection providers for the following databases:

◦ Protobuf data

(sql-data-

sources-

avro.html)

◦ MS SQL

◦ Oracle

◦ PostgreSQL

◦ DB2

◦ MariaDB

◦ Protobuf

◦ MS SQL

◦ Oracle

◦ PostgreSQL

◦ DB2

◦ MariaDB

◦ Protobuf

If the requirements are not met, please consider using the `JdbcConnectionProvider` developer API to handle custom authentication.



(index.html)3.5.1

Python

Scala

Java

R

SQL

Spark SQL Guide (sql-programming-guide.html)

```
# Note: JDBC loading and saving can be achieved via either
the load/save or jdbc methods
# Loading data from a JDBC source
jdbcDF = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql:dbserver") \
    .option("dbtable", "schema.tablename") \
    .option("user", "username") \
    .option("password", "password") \
    .load()
# Saving data to a JDBC source
jdbcDF.write \
    .format("jdbc") \
    .option("url", "jdbc:postgresql:dbserver") \
    .option("dbtable", "schema.tablename") \
    .option("user", "username") \
    .option("password", "password") \
    .save()
# Specifying create table column data types on read
jdbcDF = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql:dbserver") \
    .option("dbtable", "schema.tablename") \
    .option("user", "username") \
    .option("password", "password") \
    .option("customSchema", "id DECIMAL(38, 0), name
STRING") \
    .load()
# Specifying create table column data types on write
jdbcDF.write \
    .format("jdbc") \
    .option("url", "jdbc:postgresql:dbserver") \
    .option("dbtable", "schema.tablename") \
    .option("user", "username") \
    .option("password", "password") \
    .option("createTableColumnTypes", "name CHAR(64),
comments VARCHAR(1024)") \
    .save()
# Hive tables
jdbcDF = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql:dbserver") \
    .option("dbtable", "schema.tablename") \
    .option("user", "username") \
    .option("password", "password") \
    .load()
# Avro files
jdbcDF = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql:dbserver") \
    .option("dbtable", "schema.tablename") \
    .option("user", "username") \
    .option("password", "password") \
    .load()
# Protobuf data
jdbcDF = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:postgresql:dbserver") \
    .option("dbtable", "schema.tablename") \
    .option("user", "username") \
    .option("password", "password") \
    .load()
```

Find full example code at "examples/src/main/python/sql/datasource.py" in the Spark repo.

- Data Sources (sql-data-sources-jdbc.html)
- Avro Files (sql-data-sources-avro.html)
- Protobuf data (sql-data-sources-protobuf.html)